

Estruturas de Dados: Organizando a Informação

Instrutor: Djalma Batista Barbosa Junior



Introdução: A Arquitetura da Informação

Bem-vindo ao estudo focado em um dos pilares mais cruciais da ciência da computação: as Estruturas de Dados. À medida que os problemas se tornam mais complexos, a necessidade de organizar, armazenar e manipular grandes volumes de dados de forma eficiente torna-se a principal preocupação de um desenvolvedor. As estruturas de dados são, em essência, os esquemas e modelos que nos permitem construir a arquitetura da informação dentro de um programa.

Nesta apostila, vamos nos aprofundar nas maneiras fundamentais de organizar dados. Começaremos com as estruturas estáticas, como vetores e matrizes, que formam a base para coleções de dados do mesmo tipo, e registros, que nos permitem agrupar informações de tipos diferentes. Em seguida, exploraremos estruturas com disciplinas de acesso específicas, como Pilhas e Filas, que são definidas não apenas por como armazenam dados, mas pelas regras de como esses dados podem ser acessados. Por fim, abordaremos algoritmos essenciais de ordenação e busca, que são as ferramentas que dão vida a essas estruturas, permitindo-nos processar a informação de maneira eficaz. O domínio desses conceitos é o que diferencia a simples codificação da engenharia de software robusta e eficiente.

Capítulo 1: Estruturas de Dados Estáticas: Organizando Informações

À medida que os problemas se tornam mais complexos, a necessidade de organizar e armazenar grandes volumes de dados de forma eficiente se torna crucial. Estruturas de dados são maneiras de organizar e armazenar informações na memória do computador. Este capítulo foca nas estruturas estáticas, cujo tamanho é geralmente definido no início do algoritmo. A principal distinção conceitual aqui é entre estruturas **homogêneas**, que armazenam múltiplos itens do mesmo tipo (vetores e matrizes), e estruturas **heterogêneas**, que agrupam itens de tipos diferentes em uma única entidade (registros).

1.1 Vetores (Arrays Unidimensionais)

Imagine que você precisa armazenar a nota de 50 alunos. Sem uma estrutura de dados, você teria que criar 50 variáveis diferentes: `nota1`, `nota2`, `nota3`, e assim por diante.¹ Isso seria impraticável.

Um **vetor** (ou array unidimensional) resolve esse problema. É uma estrutura de dados que armazena uma coleção de elementos **do mesmo tipo** em posições de memória contíguas. Cada elemento na coleção é acessado por um

índice, que é um número inteiro que indica sua posição no vetor. Na maioria das linguagens, a indexação começa em 0, mas em pseudocódigo, é comum usar 1 como índice inicial para fins didáticos.

Declaração e Acesso em Pseudocódigo:

```
// Declara um vetor chamado 'notas' com 50 posições para números reais
```

```
VAR
```

```
notas: VETOR[1..50] DE REAL
```

```
i: INTEIRO
```

```
// Para atribuir um valor à primeira posição
```

```
notas <- 8.5
```

```
// Para ler o valor da terceira posição
```

```
ESCREVA notas
```

O uso de laços PARA é o padrão idiomático para percorrer e processar todos os elementos de um vetor, pois a estrutura contada do laço mapeia perfeitamente a estrutura indexada do vetor.

Exemplo: Ler 5 notas e armazená-las em um vetor.

```
PARA i DE 1 ATÉ 5 FAÇA
```

```
ESCREVA "Digite a nota ", i, ": "
```

```
LEIA notas[i]
```

```
FIM_PARA
```

1.2 Matrizes (Arrays Bidimensionais)

Uma **matriz** é uma extensão do conceito de vetor para duas dimensões. Pode ser visualizada como uma tabela ou grade, com linhas e colunas. Tecnicamente, uma matriz é um "vetor de vetores". Ela é ideal para representar dados que têm uma estrutura tabular, como um tabuleiro de jogo, uma planilha ou uma imagem pixelada.

Cada elemento em uma matriz é acessado por dois índices: um para a linha e outro para a coluna.

Declaração e Acesso em Pseudocódigo:

```
// Declara uma matriz 3x3 para um jogo da velha
```

```
VAR
```

```
    tabuleiro: VETOR[1..3, 1..3] DE CARACTERE
```

```
    linha, coluna: INTEIRO
```

```
// Para marcar um 'X' no centro do tabuleiro (linha 2, coluna 2)
```

```
tabuleiro <- 'X'
```

Para percorrer uma matriz, o padrão é utilizar laços PARA aninhados: um laço externo para as linhas e um laço interno para as colunas.

Exemplo: Preencher uma matriz 2x2 com dados do usuário.

```
PARA linha DE 1 ATÉ 2 FAÇA
  PARA coluna DE 1 ATÉ 2 FAÇA
    ESCRIVA "Digite o valor para a posição [", linha, ",", coluna, "]: "
    LEIA matriz[linha, coluna]
  FIM_PARA
FIM_PARA
```

1.3 Registros (Structs)

Enquanto vetores e matrizes são homogêneos, os **registros** (ou *structs*) são estruturas de dados **heterogêneas**. Eles permitem agrupar múltiplos campos de dados de **tipos diferentes** sob um único nome, representando uma entidade do mundo real.

Pense em um cadastro de aluno. Um aluno possui um nome (texto), uma idade (inteiro) e uma média (real). Essas informações de tipos diferentes estão logicamente conectadas. Um registro permite encapsular todos esses dados em uma única variável aluno.

Definição e Uso em Pseudocódigo:

```
// Primeiro, define-se o TIPO de registro
TIPO TAluno = REGISTRO
  nome: CARACTERE
  idade: INTEIRO
  media: REAL
FIM_REGISTRO

// Depois, declara-se uma variável desse novo tipo
VAR
  aluno1: TAluno

// O acesso aos campos é feito com a notação de ponto (.)
aluno1.nome <- "João Silva"
```

```
aluno1.idade <- 20  
aluno1.media <- 8.7
```

```
ESCREVA "Nome do aluno: ", aluno1.nome
```

A verdadeira força dessas estruturas se manifesta quando elas são combinadas. O padrão mais comum e poderoso é criar um **vetor de registros**. Isso permite representar uma coleção de entidades complexas, como uma turma inteira de alunos, um catálogo de produtos ou uma lista de contatos.

Exemplo: Vetor de Registros para uma turma de 3 alunos.

```
VAR
```

```
  turma: VETOR[1..3] DE TAluno
```

```
  i: INTEIRO
```

```
// Preenchendo os dados do primeiro aluno
```

```
turma.nome <- "João Silva"
```

```
turma.media <- 8.7
```

```
// Usando um laço para preencher os dados de toda a turma
```

```
PARA i DE 1 ATÉ 3 FAÇA
```

```
  ESCREVA "Digite o nome do aluno ", i, ":"
```

```
  LEIA turma[i].nome
```

```
  ESCREVA "Digite a média do aluno ", i, ":"
```

```
  LEIA turma[i].media
```

```
FIM_PARA
```

Exercícios Práticos do Capítulo 1

1. **Vetor:** Crie um algoritmo que leia 10 números inteiros para um vetor. Depois, percorra o vetor e exiba apenas os números pares que foram digitados.
2. **Vetor:** Desenvolva um algoritmo que leia 5 nomes para um vetor de caracteres e, em seguida, exiba os nomes na ordem inversa em que foram digitados.
3. **Matriz:** Escreva um algoritmo que preencha uma matriz 3x3 de números inteiros. Ao final, o algoritmo deve calcular e exibir a soma de todos os elementos da matriz.
4. **Registro e Vetor de Registros:**
 - Defina um tipo de registro chamado TProduto que contenha os campos: nome (CARACTERE), preco (REAL) e estoque (INTEIRO).
 - Crie um algoritmo que declare um vetor de 3 posições do tipo TProduto.
 - O algoritmo deve pedir ao usuário para preencher os dados dos 3 produtos.
 - Ao final, deve percorrer o vetor e exibir o nome de todos os produtos que custam mais de R\$ 50.00.

Capítulo 2: Estruturas de Dados Lineares com Disciplina de Acesso

Este capítulo introduz duas estruturas de dados fundamentais, Pilha e Fila, que, embora possam ser implementadas usando vetores, são definidas não pela forma como armazenam os dados, mas pela **disciplina de acesso** que impõem.

Elas são, conceitualmente, listas com restrições intencionais sobre como os elementos podem ser inseridos e removidos. Essa restrição não é uma limitação, mas uma característica de design poderosa que modela perfeitamente problemas do mundo real onde a ordem de processamento é crucial.

2.1 Pilhas (Stacks): O Princípio LIFO

Uma **Pilha** (Stack) é uma estrutura de dados linear que segue o princípio **LIFO** (**Last-In, First-Out**), que significa "o último a entrar é o primeiro a sair".

A melhor analogia para uma pilha é uma pilha de pratos em um restaurante. Você só pode adicionar um novo prato no topo da pilha e só pode remover o prato que está no topo. É impossível remover um prato do meio da pilha sem antes remover todos os que estão acima dele.

Essa estrutura é ideal para problemas que envolvem reverter ações ou rastrear passos. Aplicações comuns incluem:

- O mecanismo de "Desfazer" (Ctrl+Z) em editores de texto, onde a última ação realizada é a primeira a ser desfeita.
- O histórico de navegação de um navegador web, onde o botão "Voltar" leva à última página visitada.
- O gerenciamento de chamadas de funções em linguagens de programação.

As operações fundamentais de uma pilha são:

- **Empilhar (Push):** Adiciona um novo elemento no **topo** da pilha.
- **Desempilhar (Pop):** Remove e retorna o elemento que está no **topo** da pilha.
- **Topo (Top ou Peek):** Retorna o elemento do topo sem removê-lo.
- **Vazia (IsEmpty):** Verifica se a pilha não contém elementos.

2.2 Filas (Queues): O Princípio FIFO

Uma **Fila** (Queue) é uma estrutura de dados linear que segue o princípio **FIFO** (**First-In, First-Out**), que significa "o primeiro a entrar é o primeiro a sair".

A analogia perfeita é uma fila de pessoas em um caixa de supermercado ou banco. A primeira pessoa a chegar na fila é a primeira a ser atendida. Novas pessoas sempre entram no final da fila.

Essa estrutura é a escolha natural para problemas que envolvem o processamento de tarefas ou eventos na ordem em que eles chegam. Aplicações comuns incluem:

- Gerenciamento de filas de impressão, onde os documentos são impressos na ordem em que foram enviados.
- Sistemas de atendimento ao cliente (call centers, chats).
- Processamento de tarefas em sistemas operacionais.

As operações fundamentais de uma fila são:

- **Enfileirar (Enqueue):** Adiciona um novo elemento no **final** da fila.
- **Desenfileirar (Dequeue):** Remove e retorna o elemento do **início** da fila.
- **Início (Front ou Peek):** Retorna o primeiro elemento da fila sem removê-lo.
- **Vazia (IsEmpty):** Verifica se a fila não contém elementos.

A escolha entre Pilha e Fila é inteiramente ditada pela natureza do problema. A análise do requisito de "ordem de processamento" é a chave para a seleção correta da estrutura.

Tabela 2.1 - Comparativo: Pilha vs. Fila

Característica	Pilha (Stack)	Fila (Queue)
Princípio	LIFO (Last-In, First-Out)	FIFO (First-In, First-Out)
Operação de Inserção	Empilhar (Push)	Enfileirar (Enqueue)
Local da Inserção	Topo	Final
Operação de Remoção	Desempilhar (Pop)	Desenfileirar (Dequeue)
Local da Remoção	Topo	Início
Analogia do Mundo Real	Pilha de pratos, botão "Voltar" do navegador	Fila de banco, fila de impressão

Exercícios Práticos do Capítulo 2

1. **Pilha:** Usando as operações Empilhar, Desempilhar e Vazia, escreva um pseudocódigo que leia uma palavra do usuário e a imprima de trás para frente. (Dica: empilhe cada letra da palavra e depois desempilhe para imprimir).
2. **Fila:** Simule em pseudocódigo o atendimento em uma pequena clínica. O algoritmo deve permitir:
 - Adicionar um paciente à fila (Enfileirar).
 - Chamar o próximo paciente para atendimento (Desenfileirar).
 - O programa deve rodar em um laço, mostrando um menu com essas opções, até que o usuário escolha "Sair".
3. **Conceitual:** Para cada um dos cenários a seguir, indique se uma Pilha ou uma Fila seria a estrutura de dados mais apropriada e justifique sua resposta:
 - A. Um sistema que gerencia as tarefas a serem executadas por um processador, na ordem em que foram solicitadas.
 - B. Um programa para verificar se uma expressão matemática tem parênteses, chaves e colchetes balanceados (ex: [({ })] está balanceado, [(]) não está).
 - C. Um sistema de logística onde os últimos contêineres carregados em um navio são os primeiros a serem descarregados no porto.

Capítulo 3: Algoritmos Fundamentais de Ordenação e Busca

Manipular coleções de dados, como vetores, frequentemente envolve duas tarefas essenciais: ordená-los e buscar por informações específicas dentro deles. Este capítulo apresenta algoritmos clássicos para essas duas operações. Compreender esses algoritmos não apenas fornece ferramentas práticas, mas também introduz conceitos importantes como eficiência e pré-condições algorítmicas.

3.1 Ordenação de Dados: O Algoritmo Bubble Sort

Ordenar significa arranjar os elementos de uma coleção em uma sequência específica, geralmente crescente ou decrescente. Ter dados ordenados pode otimizar drasticamente a busca por informações, como veremos a seguir.

O **Bubble Sort** (Ordenação por Bolha) é um dos algoritmos de ordenação mais simples de entender, sendo um excelente ponto de partida para o estudo do tema. Sua lógica consiste em percorrer o vetor repetidamente, comparando cada par de elementos adjacentes e trocando-os de posição se estiverem na ordem errada.

A cada passagem completa pelo vetor, o maior elemento "borbulha" para sua posição correta no final da lista. O processo é repetido até que nenhuma troca seja necessária em uma passagem completa, indicando que o vetor está ordenado.

Passo a Passo do Bubble Sort (Vetor: ``)

1ª Passagem:

- Compara 5 e 1. Troca. Vetor: ``
- Compara 5 e 4. Troca. Vetor: ``
- Compara 5 e 2. Troca. Vetor: ``
- *Ao final da 1ª passagem, o maior elemento (5) está na posição correta.*

• 2ª Passagem:

- Compara 1 e 4. Não troca. Vetor: ``
- Compara 4 e 2. Troca. Vetor: ``
- *Ao final da 2ª passagem, o segundo maior elemento (4) está na posição correta.*

• 3ª Passagem:

- Compara 1 e 2. Não troca. Vetor: ``
- *Nenhuma troca ocorreu. O algoritmo pode parar, pois o vetor está ordenado.*

Pseudocódigo do Bubble Sort:

ALGORITMO BubbleSort(V: VETOR DE INTEIROS)

VAR

i, j, temp: INTEIRO

n: INTEIRO

trocou: LOGICO

n <- tamanho_do_vetor(V)

REPITA

trocou <- FALSO

PARA i DE 1 ATÉ n-1 FAÇA

SE V[i] > V[i+1] ENTÃO

// Troca os elementos de posição

temp <- V[i]

V[i] <- V[i+1]

V[i+1] <- temp

trocou <- VERDADEIRO

```
FIM_SE  
FIM_PARA  
n <- n - 1 // Otimização: o último elemento já está no lugar certo  
ATÉ trocou == FALSO  
FIM_ALGORITMO
```

3.2 Busca de Dados

Buscar é o processo de encontrar a localização de um elemento específico (a "chave") dentro de uma coleção.

3.2.1 Busca Linear (Sequencial)

A **Busca Linear** é o método mais intuitivo. O algoritmo percorre a coleção do início ao fim, um elemento por vez, comparando cada um com a chave de busca. Se o elemento for encontrado, o algoritmo retorna sua posição (índice). Se o final da coleção for alcançado sem encontrar o elemento, o algoritmo indica que a busca falhou (geralmente retornando um valor como -1).

- **Vantagem:** É simples de implementar e, crucialmente, **funciona em vetores desordenados**.
- **Desvantagem:** É ineficiente para grandes coleções de dados. No pior caso (o elemento não existe), é necessário verificar todos os elementos.

Pseudocódigo da Busca Linear:

```
FUNÇÃO BuscaLinear(V: VETOR DE INTEIROS, chave: INTEIRO): INTEIRO
VAR
  i: INTEIRO
INICIO
  PARA i DE 1 ATÉ tamanho_do_vetor(V) FAÇA
    SE V[i] == chave ENTÃO
      RETORNE i // Retorna o índice onde o elemento foi encontrado
    FIM_SE
  FIM_PARA
  RETORNE -1 // Retorna -1 se o elemento não for encontrado
FIM_FUNÇÃO
```

3.2.2 Busca Binária

A **Busca Binária** é um algoritmo de busca muito mais eficiente, mas com uma **pré-condição** fundamental: o vetor **deve estar ordenado**.²⁰ A violação dessa pré-condição não apenas torna o algoritmo ineficiente, mas também produzirá resultados incorretos.

O algoritmo segue uma estratégia de "divisão e conquista":

1. Ele examina o elemento no meio do vetor.
2. Se este elemento for a chave, a busca termina com sucesso.
3. Se a chave for menor que o elemento do meio, o algoritmo sabe que, se a chave existir, ela deve estar na primeira metade do vetor. A segunda metade é então descartada.
4. Se a chave for maior que o elemento do meio, a busca continua na segunda metade, e a primeira é descartada.
5. O processo é repetido na metade restante, dividindo o espaço de busca ao meio

a cada passo, até que o elemento seja encontrado ou o espaço de busca se esgote.

Essa abordagem revela um importante **trade-off** (custo-benefício) na computação. A Busca Binária é muito mais rápida que a Linear, mas exige o custo inicial de ordenar o vetor. A decisão de qual usar depende da aplicação: se os dados são buscados com frequência, mas modificados raramente, vale a pena pagar o preço da ordenação para se beneficiar de buscas mais rápidas.

Pseudocódigo da Busca Binária:

FUNÇÃO BuscaBinaria(V: VETOR ORDENADO DE INTEIROS, chave: INTEIRO):
INTEIRO

VAR

início, fim, meio: INTEIRO

INICIO

início <- 1

fim <- tamanho_do_vetor(V)

ENQUANTO início <= fim FAÇA

meio <- (início + fim) / 2 // Calcula o índice do meio

SE V[meio] == chave ENTÃO

RETORNE meio // Elemento encontrado

SENÃO SE V[meio] < chave ENTÃO

início <- meio + 1 // Busca na metade direita

SENÃO

fim <- meio - 1 // Busca na metade esquerda

FIM_SE

FIM_ENQUANTO

RETORNE -1 // Elemento não encontrado

FIM_FUNÇÃO

Exercícios Práticos do Capítulo 3

1. **Bubble Sort:** Mostre o estado do vetor `` após cada **passagem completa** do algoritmo Bubble Sort.
2. **Busca Linear:** Dado o vetor ["maçã", "banana", "uva", "pêra", "laranja"], quantos elementos a Busca Linear precisaria verificar, no pior caso, para determinar que a fruta "abacaxi" não está no vetor?
3. **Busca Binária:** Dado o vetor ordenado `` , descreva os passos (quais elementos do meio são verificados) que a Busca Binária realizaria para encontrar o número 20.
4. **Algoritmo Combinado:** Crie um algoritmo completo em pseudocódigo que:
 - Leia 8 números inteiros para um vetor.
 - Ordene o vetor usando o algoritmo Bubble Sort.
 - Exiba o vetor ordenado.
 - Peça ao usuário para digitar um número a ser buscado.
 - Use a Busca Binária para encontrar o número e exiba uma mensagem informando se o número foi encontrado e em qual posição.

Apêndice: Soluções dos Exercícios Seleccionados

Esta seção fornece as soluções para exercícios seleccionados de cada capítulo, permitindo que o aluno verifique seu trabalho e compreenda a aplicação correta dos conceitos.

Capítulo 1 - Exercício 4 (Vetor de Registros de Produtos):

ALGORITMO "CadastroProdutos"

TIPO

TProduto = REGISTRO

nome: CARACTERE

preco: REAL

estoque: INTEIRO

FIM_REGISTRO

VAR

catalogo: VETOR[1..3] DE TProduto

i: INTEIRO

INICIO

// Preenchimento dos dados

PARA i DE 1 ATÉ 3 FAÇA

ESCREVA "--- Produto ", i, " ---"

ESCREVA "Nome: "

LEIA catalogo[i].nome

```
    ESCREVA "Preço: "  
    LEIA catalogo[i].preco  
    ESCREVA "Estoque: "  
    LEIA catalogo[i].estoque  
FIM_PARA  
  
// Exibição dos produtos com preço > 50.00  
ESCREVA "--- Produtos com preço superior a R$ 50.00 ---"  
PARA i DE 1 ATÉ 3 FAÇA  
    SE catalogo[i].preco > 50.00 ENTÃO  
        ESCREVA catalogo[i].nome  
    FIM_SE  
FIM_PARA  
FIM_ALGORITMO
```

Capítulo 3 - Exercício 4 (Algoritmo Combinado de Ordenação e Busca):

```
ALGORITMO "OrdenaBusca"  
VAR  
    vetor_numeros: VETOR[1..8] DE INTEIRO  
    i, j, temp, chave, resultado_busca: INTEIRO  
    trocou: LOGICO  
    inicio, fim, meio: INTEIRO  
  
INICIO  
    // Leitura dos números  
    PARA i DE 1 ATÉ 8 FAÇA  
        ESCREVA "Digite o ", i, "º número: "  
        LEIA vetor_numeros[i]  
    FIM_PARA  
  
    // Ordenação com Bubble Sort  
    REPITA  
        trocou <- FALSO  
        PARA i DE 1 ATÉ 7 FAÇA  
            SE vetor_numeros[i] > vetor_numeros[i+1] ENTÃO  
                temp <- vetor_numeros[i]
```

```
vetor_numeros[i] <- vetor_numeros[i+1]
vetor_numeros[i+1] <- temp
trocou <- VERDADEIRO
FIM_SE
FIM_PARA
ATÉ trocou == FALSO

// Exibição do vetor ordenado
ESCREVA "Vetor ordenado: "
PARA i DE 1 ATÉ 8 FAÇA
    ESCREVA vetor_numeros[i], " "
FIM_PARA

// Busca Binária
ESCREVA "Digite um número para buscar no vetor: "
LEIA chave

inicio <- 1
fim <- 8
resultado_busca <- -1
ENQUANTO (inicio <= fim) E (resultado_busca == -1) FAÇA
    meio <- (inicio + fim) / 2
    SE vetor_numeros[meio] == chave ENTÃO
        resultado_busca <- meio
    SENÃO SE vetor_numeros[meio] < chave ENTÃO
        inicio <- meio + 1
    SENÃO
        fim <- meio - 1
    FIM_SE
FIM_ENQUANTO

// Exibição do resultado da busca
SE resultado_busca <> -1 ENTÃO
    ESCREVA "O número ", chave, " foi encontrado na posição ",
resultado_busca, "."
SENÃO
    ESCREVA "O número ", chave, " não foi encontrado no vetor."
FIM_SE
FIM_ALGORITMO
```

